# Module 9: AI Agent Observability and AgentOPs

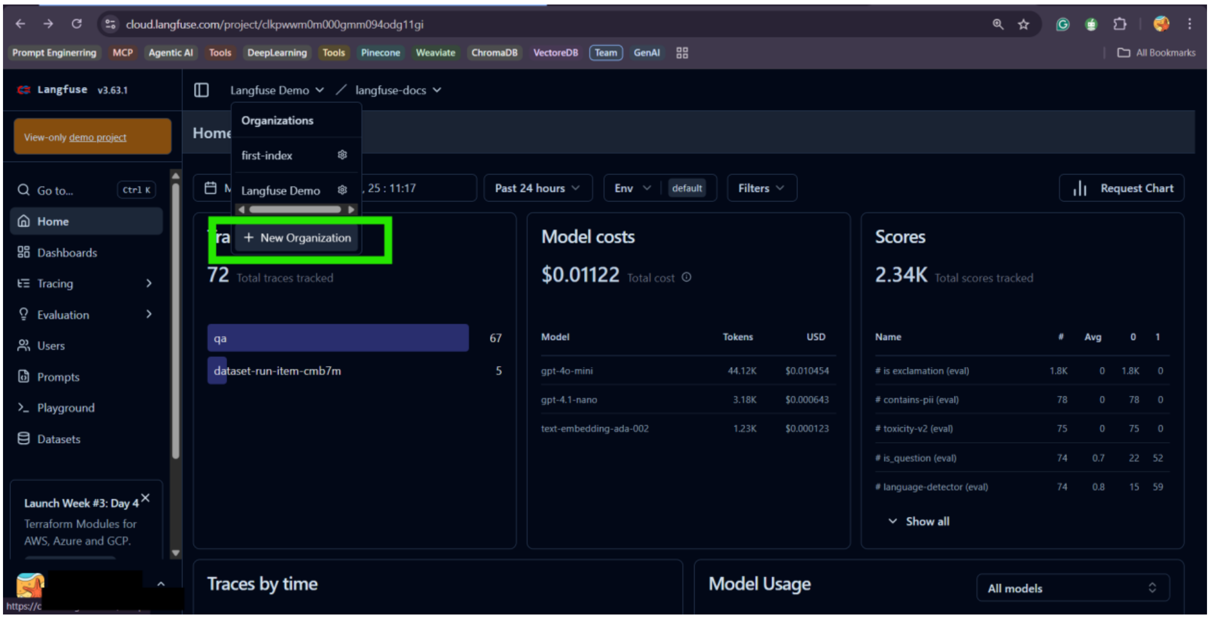
## Demo-3

# Demo-1: AI Observability with Langfuse

[Langfuse](https://cloud.langfuse.com/): [https://cloud.langfuse.com/](https://cloud.langfuse.com/)

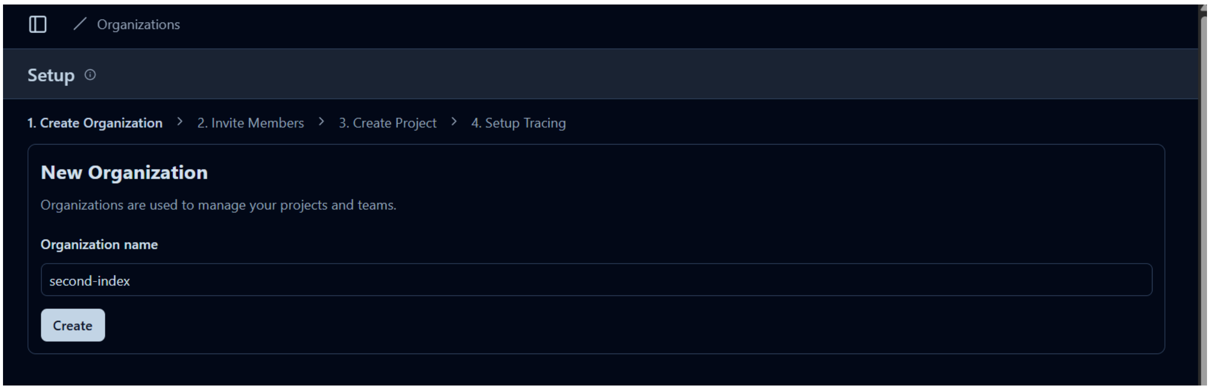
### Langfuse Account Set-up

- Sign-up or log-in to your Langfuse Account.
- Once in, Click on “New Organization” to create one.



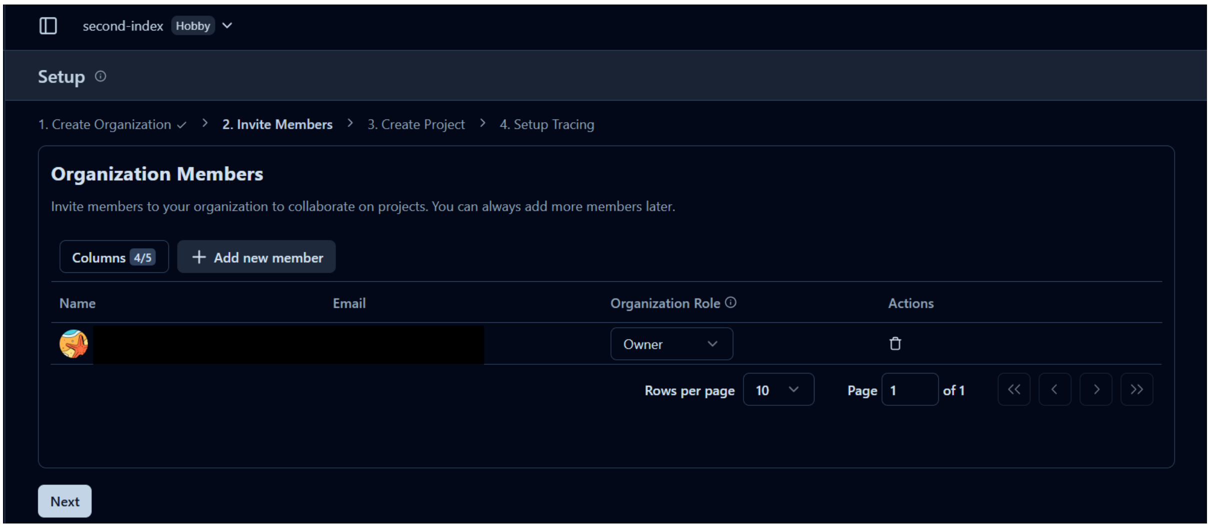

### Give a name for your Organization

- Although in the screenshot of the Langfuse interface you can see the name as second-index, you can name your organization anything.
- Going forward you will notice that the name of the organization in the demo is “first-index”



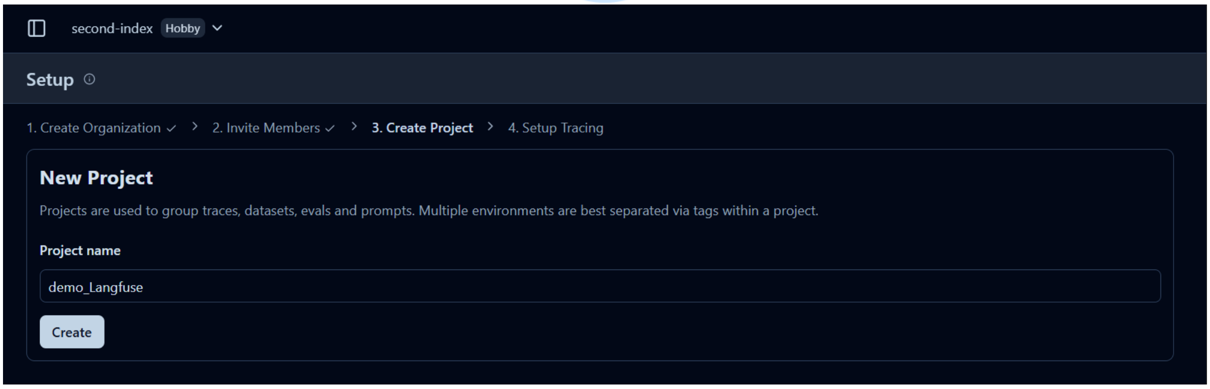

 

## Organization Members



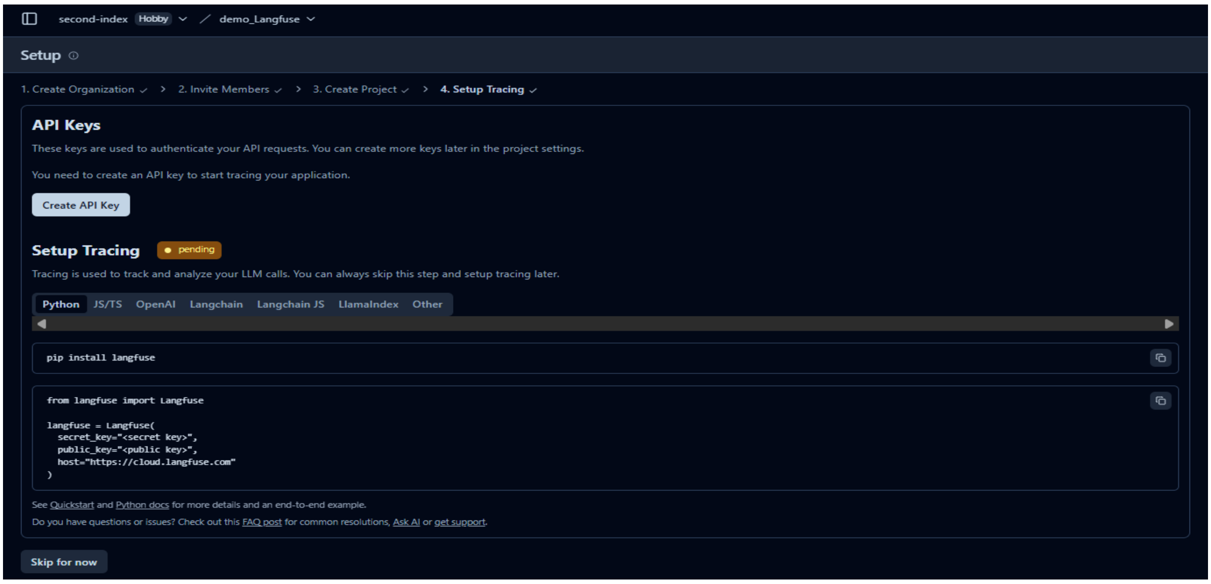

**New Project:** Give a name to your project just like you created the organization name.

- Although in the screenshot of the Langfuse interface you can see the name as demo_Langfuse, you can name your project based on your preference.
- Going forward you will notice that the name of the project in the demo is "Langfuse_demo"

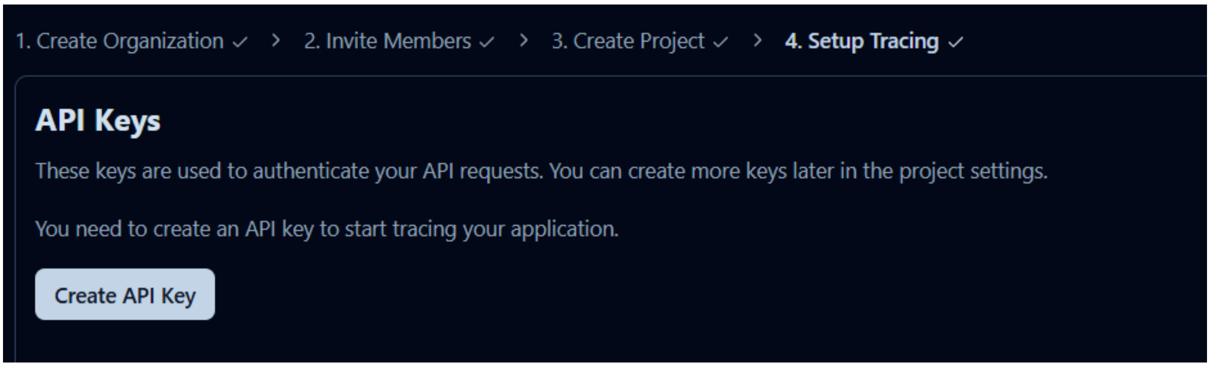


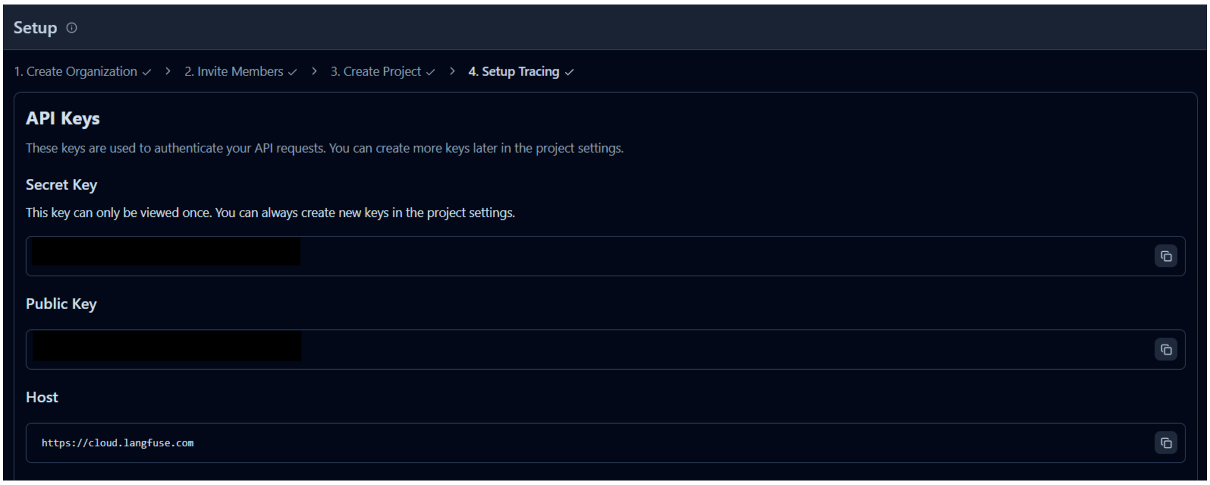
## Setup Tracing




## Create API Keys

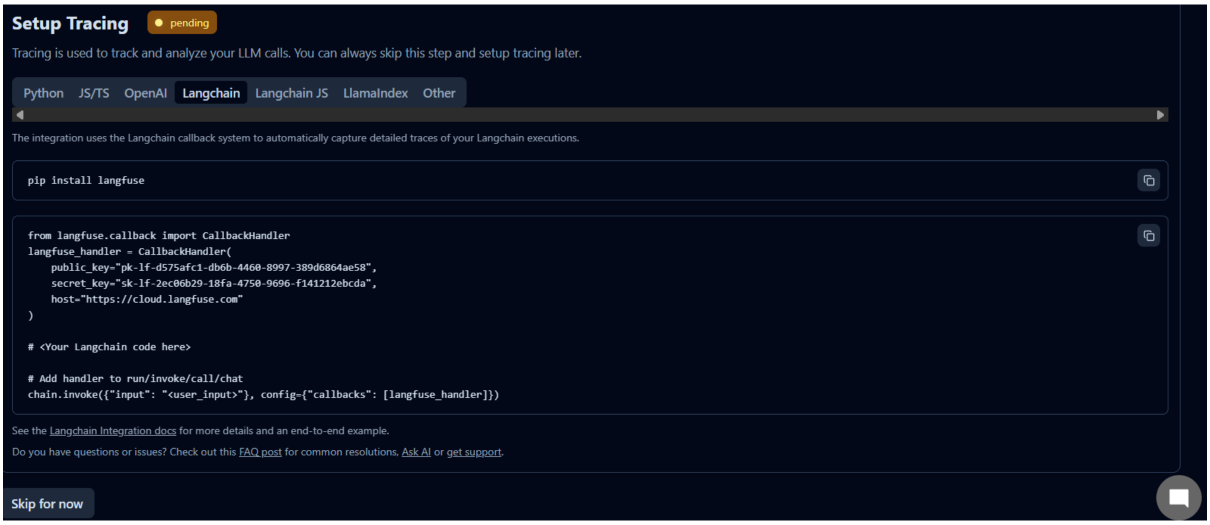



## Store your API Keys safely



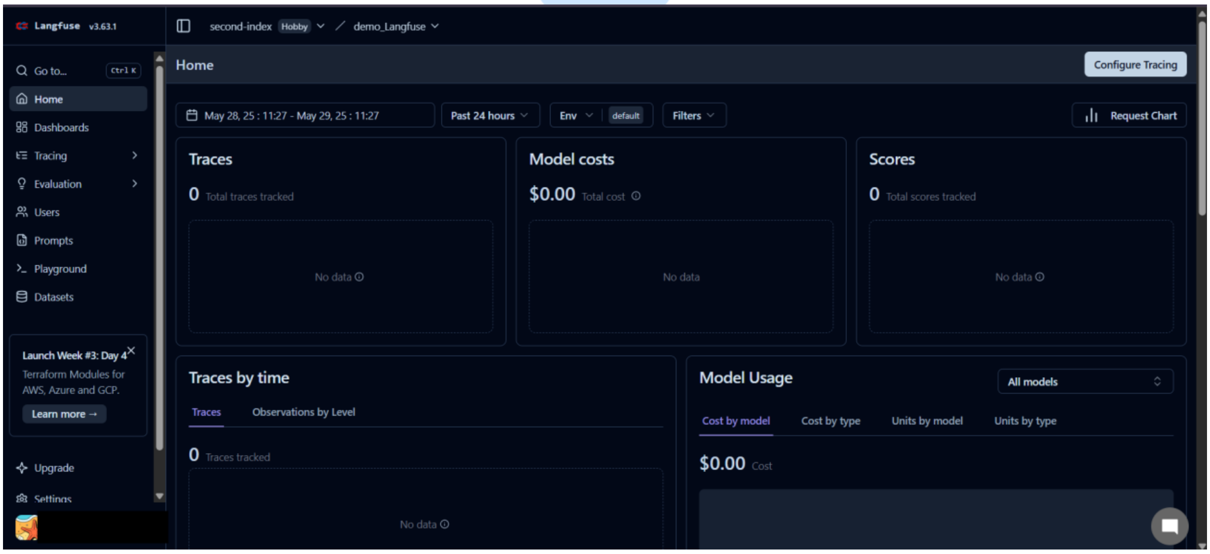
You will notice the “pending” indicator. Once you set-up your tracing in the application you want to trace, it turns to “active” and green colour indicator.

You can keep the code for guidance and then skip this step for now.




## Project Setup complete

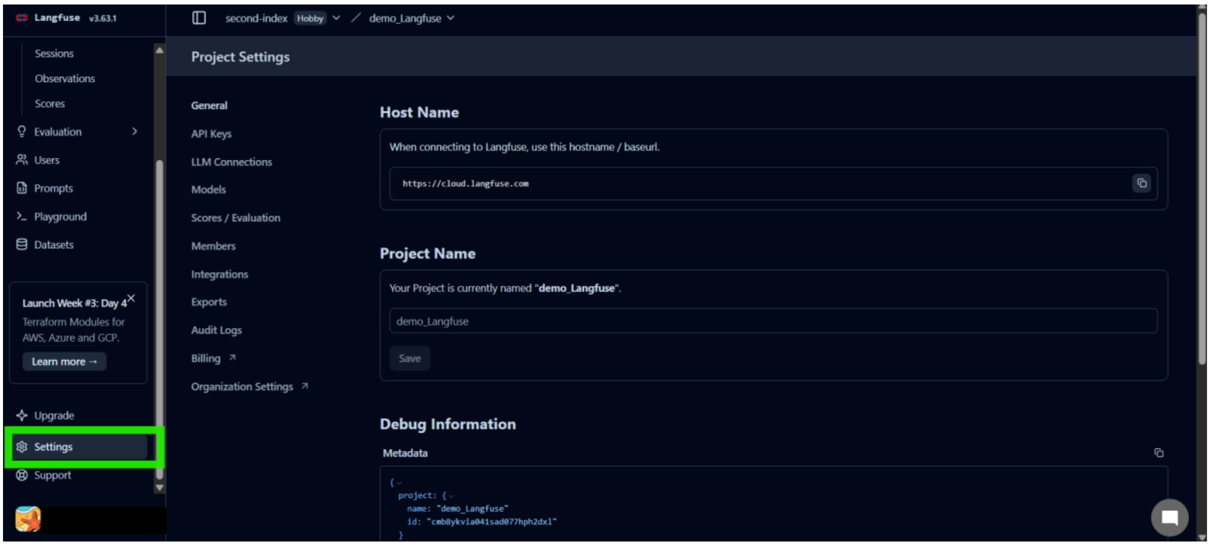
Once you are done setting up the project the Home tab will look like this.




## Settings

You can view the different Project Settings here.





## Overview

This demonstration showcases how to implement AI Observability using Langfuse in a multi-agent research application. The application leverages LangGraph to orchestrate a workflow of agents that collaboratively research a user-provided topic. The agents include a Search Agent (using `DuckDuckGo`), an Analyzer Agent, and a Summarizer Agent (both powered by the Groq API's `llama-3.3-70b-versatile model`). Langfuse is integrated to provide observability, enabling tracing, monitoring, and evaluation of the agents' performance. This setup ensures transparency into the AI system's behavior, making it easier to debug, evaluate, and improve the application. The demo runs in Google Colab and includes dynamic user input for research topics.

## Tasks Achieved

- Multi-Agent Workflow: Built a LangGraph-based workflow with three agents:
  - Search Agent: Fetches recent developments on a topic using DuckDuckGo.
  - Analyzer Agent: Analyzes search results to identify key points and trends.

- Summarizer Agent: Summarizes the analysis into a concise research report.

- AI Observability with Langfuse:
    - Traced the execution of each agent, capturing inputs, outputs, and metadata.
    - Monitored token usage for Groq API calls.
    - Evaluated the final summary with a static score and comment.
- Dynamic Input: Allowed users to input a research topic interactively in Colab.
- Error Handling: Included basic error handling for API key retrieval, Groq API calls, and DuckDuckGo searches.

**Project Structure**

The project is a single Jupyter Notebook (Langfuse_demo.ipynb) with the following structure:

- Setup: Installs dependencies and imports libraries.
- API Key Retrieval: Retrieves API keys from Colab Secrets.
- Agent Definitions: Defines the Search, Analyzer, and Summarizer agents using LangGraph.
- Workflow Compilation: Sets up the LangGraph workflow with Langfuse tracing.
- Main Function: Executes the workflow, processes user input, and logs results with Langfuse.
- Observability Integration: Uses Langfuse to trace and evaluate the workflow.

## PROJECT EXPLANATION

### Setup

To run this demo, follow these steps to set up your environment and configure Langfuse for observability.

**Step 1: Create a Google Colab Notebook:**

- Open Google Colab and create a new notebook.
- Copy the code from the provided Langfuse_demo.ipynb

**Step 2: Install Dependencies:** The notebook installs required packages (`langfuse`, `langgraph`, `langchain`, `langchain-community`, `groq`, `requests`, `duckduckgo-search`) automatically via **`!pip install`**.

**Step 3: Set Up API Keys in Colab Secrets**

- In Google Colab, go to the “Secrets” tab (lock icon on the left sidebar).
- Add the following secrets with your own values:
    - Name: GROQ_API_KEY, Value: Your Groq API key.
    - Name: LANGFUSE_API_KEY, Value: Your Langfuse secret key.
    - Name: LANGFUSE_PUBLIC_KEY, Value: Your Langfuse public key.
    - Name: LANGFUSE_HOST, Value: Your Langfuse host URL.
- Ensure all keys are set correctly to avoid runtime errors.

**Step 4: Configure Organization, Project, and IDs**

In the code, update the following fields with your own information:

- Organization Name: Replace "first-index" with your Langfuse organization name (found in your Langfuse dashboard under account settings).
- Project Name: Replace "Langfuse_demo" with your Langfuse project name (found in your Langfuse dashboard).
- User ID: Replace "demo-user-001" with a unique user ID for your demo.

- Region (LANGFUSE_HOST): Ensure the LANGFUSE_HOST matches the region of your Langfuse deployment (e.g., https://us.cloud.langfuse.com for US region, or your self-hosted URL).

### Step 5: Run the Notebook

Execute the cell in Colab.

- When prompted, enter a research topic (e.g., "quantum computing").
- The notebook will output the search results, analysis, and summary, and log the trace to Langfuse.

### Agent Development

We have previously discussed building agents in detail:

- This demo uses LangGraph to define a multi-agent workflow with a Search Agent, Analyzer Agent, and Summarizer Agent.
- Each agent performs a specific task (searching, analyzing, summarizing), and their interactions are orchestrated via a state graph.
- Langfuse enhances this setup by providing observability into their execution.

```
Enter a research topic (e.g., AI advancements, quantum computing): quantum computing
Researching topic: quantum computing
Search Results: MIT physicists predict exotic form of matter with potential for quantum computing. New work suggests the ability to create fractionalized electrons known as non-Abeliar
Another exciting 2025 development is the debut of portable quantum computing devices. Traditionally, quantum computers are massive installations requiring ultra-cold refrigerators or v
Read the latest about the development of quantum computers. Skip to main content ... 2025 — Whether bismuth is part of a class of materials highly suitable for quantum computing and sp
Analysis: After analyzing the search results for the topic 'quantum computing', the following key points and trends have been identified:

**Key Points:**

1. **Advancements in exotic matter**: Researchers at MIT have predicted the possibility of creating fractionalized electrons known as non-Abelian anyons without a magnetic field, which
2. **Portable quantum computing devices**: A startup called SaxonQ has developed a compact quantum computer that can operate without the need for large, ultra-cold refrigerators or vac
3. **New materials for quantum computing**: Bismuth is being explored as a potential material for quantum computing and spintronics, indicating ongoing research into finding suitable m

**Trends:**

1. **Increased focus on practical applications**: The development of portable quantum computing devices and the exploration of new materials suggest a growing emphasis on making quantu
2. **R
Research Summary: Quantum computing research highlights advancements in exotic matter, portable devices, and new materials. Trends show a focus on practical applications, making quantu
Go to the Langfuse dashboard to view the trace:
URL: https://cloud.langfuse.com
Organization: first-index
Project: Langfuse_demo
Look under the 'Traces' tab for 'multi-agent-research-langgraph' and check 'Sessions' for session tracking.
```

### AI Observability with Langfuse

Langfuse is integrated to provide observability into the multi-agent system, enabling tracing, monitoring, and evaluation. Here's how it's implemented:

### Step: Langfuse Initialization

A CallbackHandler is created to enable tracing of the LangGraph workflow

```
langfuse_handler = CallbackHandler(
    secret_key=LANGFUSE_SECRET_KEY,
    public_key=LANGFUSE_PUBLIC_KEY,
    host=LANGFUSE_HOST
)
```

A Langfuse client is initialized for manual scoring:

```
langfuse = Langfuse(
    public_key=LANGFUSE_PUBLIC_KEY,
    secret_key=LANGFUSE_SECRET_KEY,
    host=LANGFUSE_HOST
)
```

These use the API keys and host URL set in Colab Secrets, which have been configured with unique values.

### Step: Tracing Agent Execution

- Each agent function (search_agent, analyzer_agent, summarizer_agent) is decorated with @observe() or @observe(as_type="generation") to enable Langfuse tracing.
- Within each agent, langfuse_context.update_current_observation logs metadata, inputs, and usage details:

```python
langfuse_context.update_current_observation(
    input=f"User input: {user_input}\nSearch results: {search_results}",
    model="llama-3.3-70b-versatile",
    metadata={"organization": "first-index", "project": "Langfuse_demo", "agent": "analyzer_agent"}
)
```

```python
langfuse_context.update_current_observation(
    usage_details={
        "input": response.usage.prompt_tokens,
        "output": response.usage.completion_tokens
    }
)
```

The metadata includes your organization and project names, which you've updated to match your Langfuse setup.

**Step: Workflow Tracing**

The LangGraph workflow is compiled with the Langfuse callback handler

```python
app = workflow.compile()
app = app.with_config({"callbacks": [langfuse_handler]})
```

This ensures all agent interactions are traced as a single workflow in Langfuse.

**Step: Creating a Trace**

After the workflow completes, a trace is created in Langfuse. The user_id, organization, and project fields should match your Langfuse configuration.

```python
trace = langfuse.trace(
    name="multi-agent-research-langgraph",
    user_id="demo-user-001",
    session_id=session_id,
    metadata={
        "organization": "first-index",
        "project": "Langfuse_demo",
        "input_tokens": len(user_input.split()),
        "output_tokens": len(result["summary"].split())
    }
)
```

**Step: Scoring and Evaluation**

- The trace is scored with a static relevance score and comment
- This score appears in the Langfuse dashboard under the trace, helping you evaluate the quality of the output.

```python
trace.score(
    name="response-quality",
    value=0.9,
    comment="The research summary was concise and informative."
)
```

**Step: Flushing Events**

langfuse.flush() ensures all tracing events are sent to the Langfuse server

**Step: Viewing Traces**

- The notebook provides instructions to view the trace in the Langfuse dashboard
- Update the URL, organization, and project names to match your Langfuse setup.

```python
print("Go to the Langfuse dashboard to view the trace:")
print("URL: https://cloud.langfuse.com")
print("Organization: first-index")
print("Project: Langfuse_demo")
print("Look under the 'Traces' tab for 'multi-agent-research-langgraph' and check 'Sessions' for session tracking.")
```

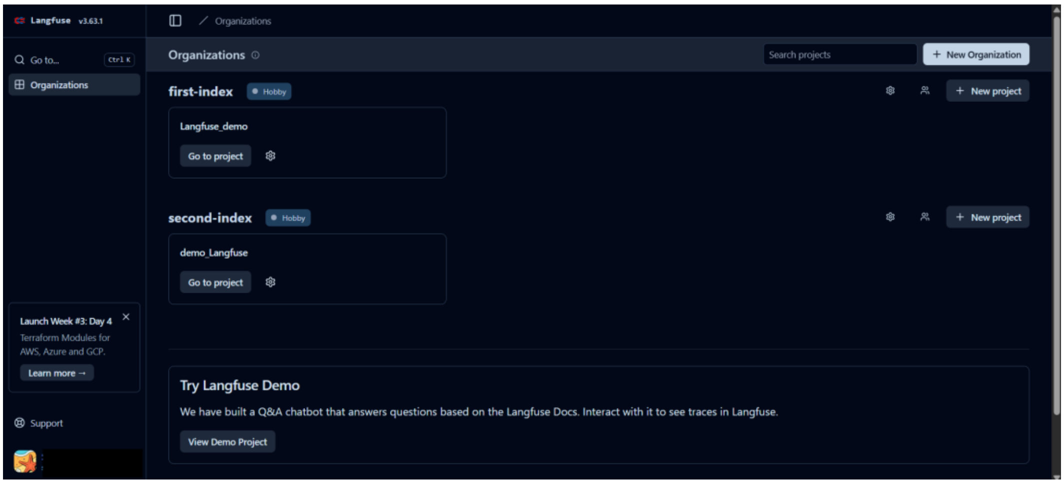
## Langfuse Interface

Click on the URL to look through the project insights

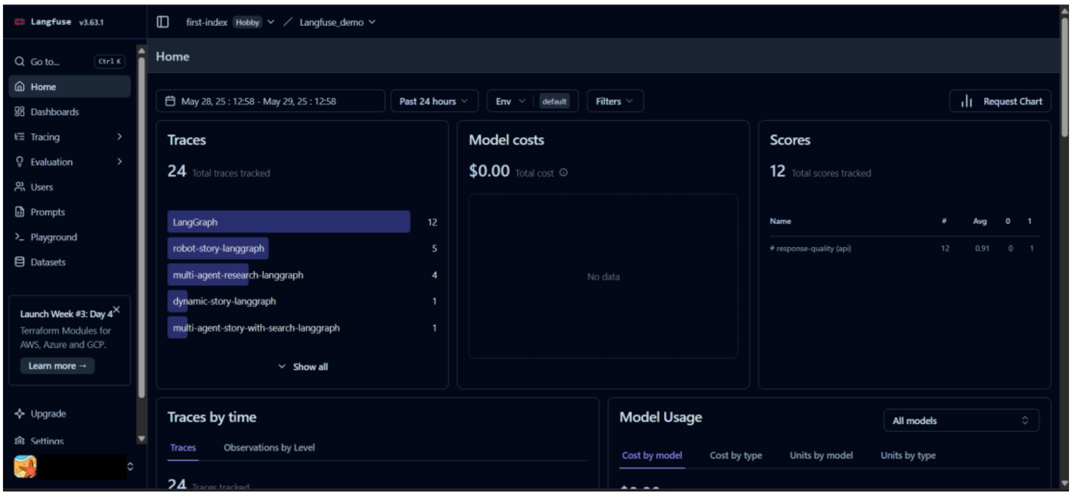
```
**Trends:**

1. **Increased focus on practical applications**:
2. **R
Research Summary: Quantum computing research high
Go to the Langfuse dashboard to view the trace:
URL: https://cloud.langfuse.com
Organization: first-index
Project: Langfuse_demo
Look under the 'Traces' tab for 'multi-agent-rese
```

You will get redirected here, to the organization's page. Click on the “Go to project” button under the designated organization for the project.



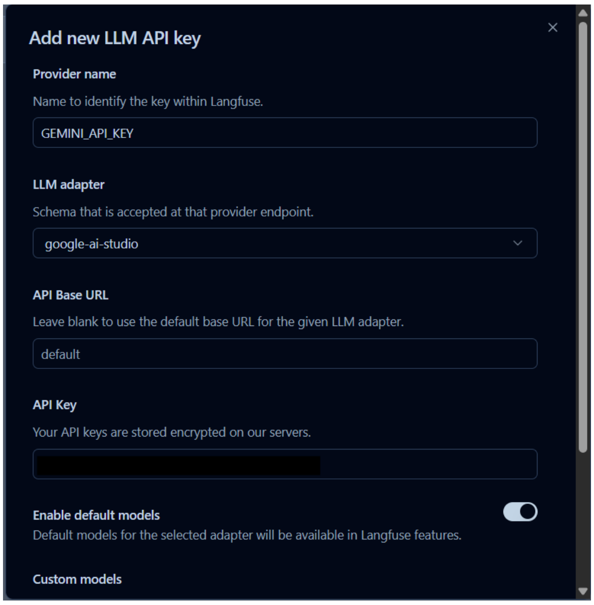

## Project Home

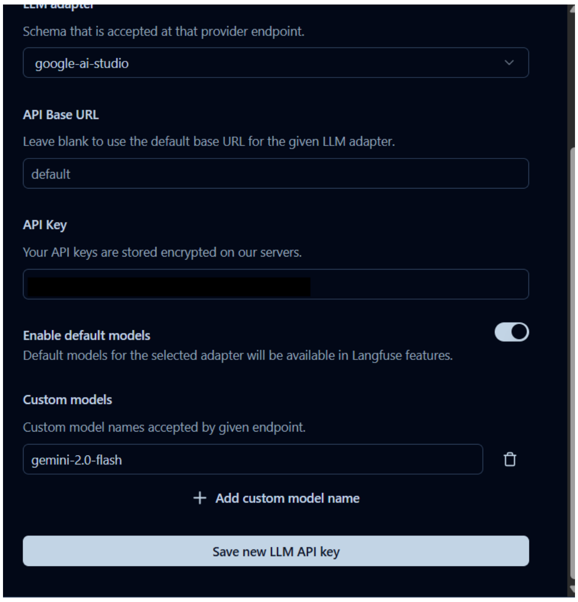


## Got to “Settings”

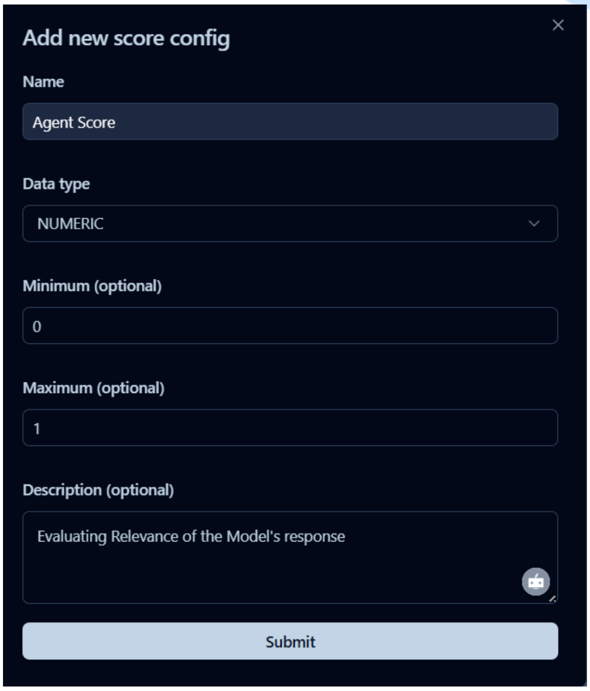
We go to the “LLM Connections” option and add a new API Key and model and save it.



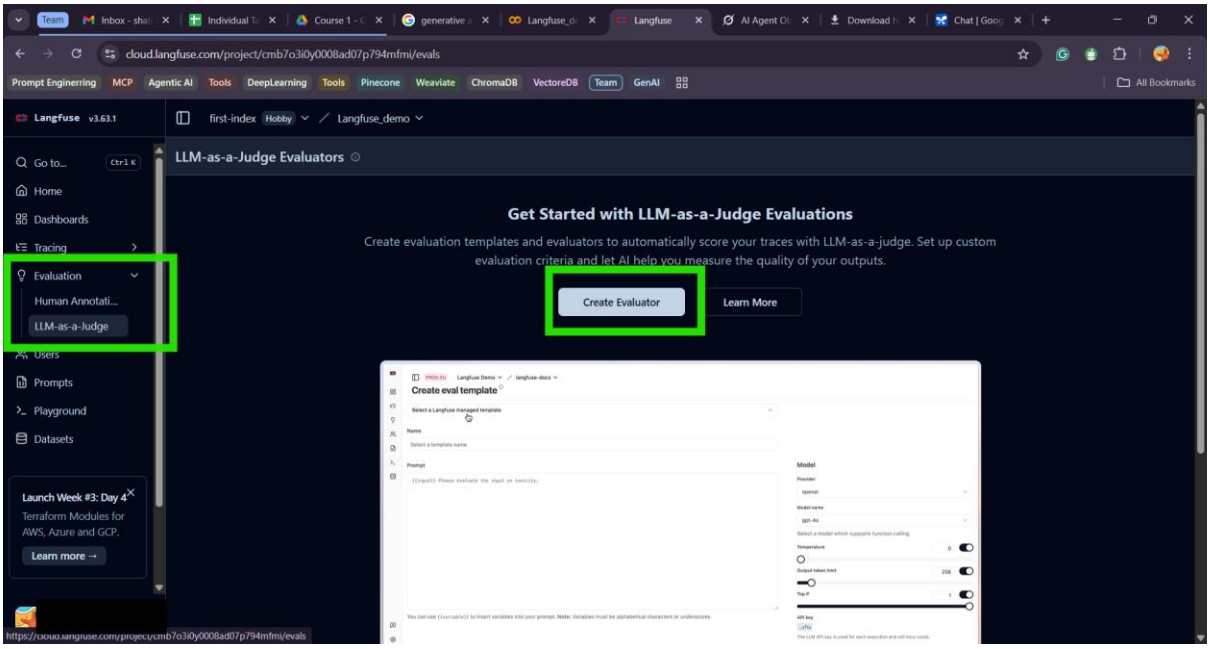

## Got to “Scores/Evaluation”

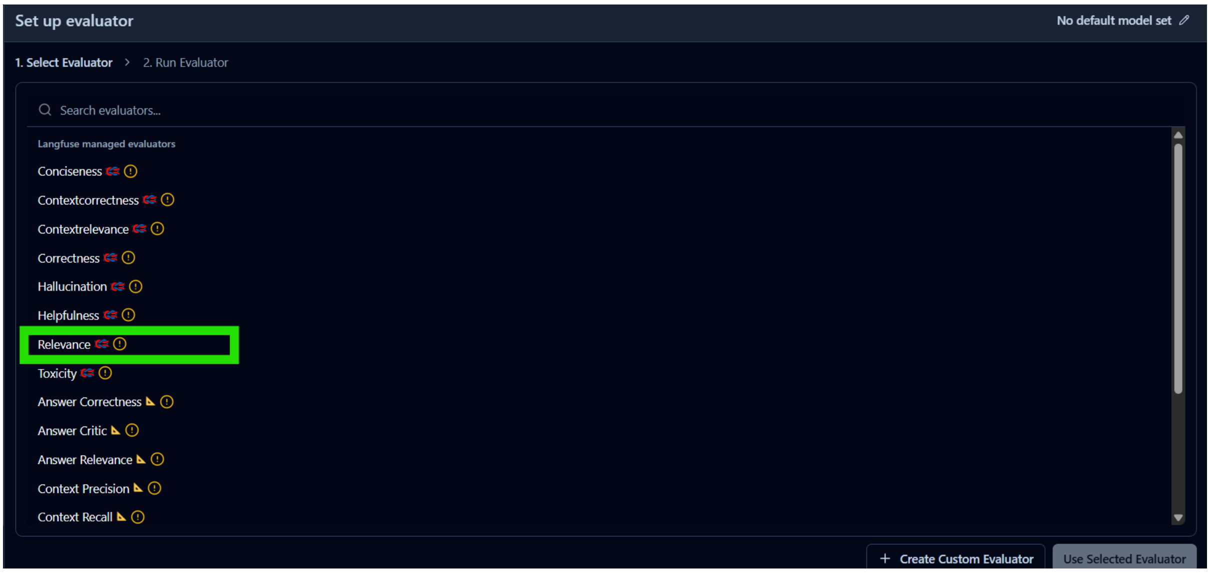
Add new score configuration



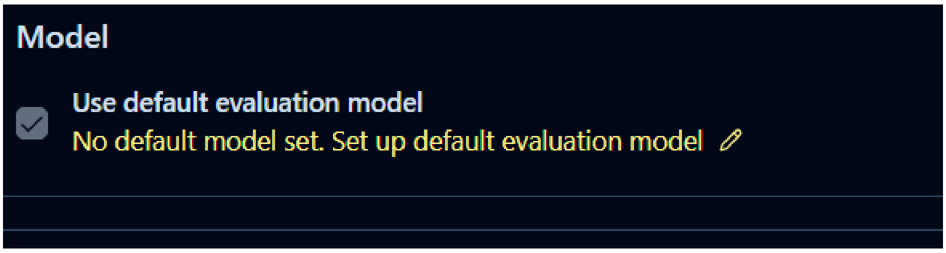

 

## Creating Evaluator

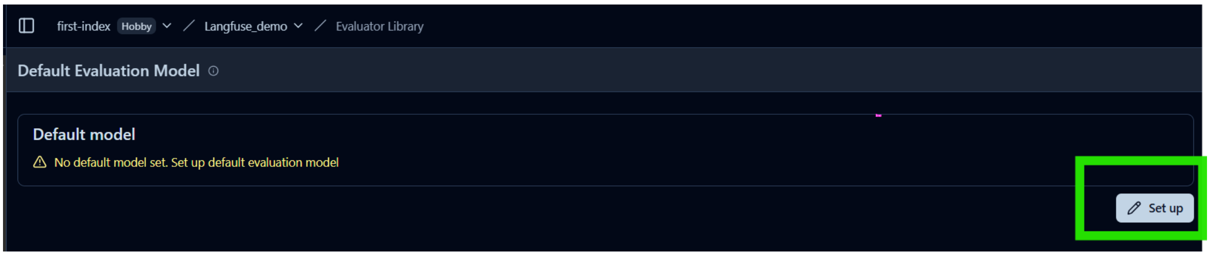


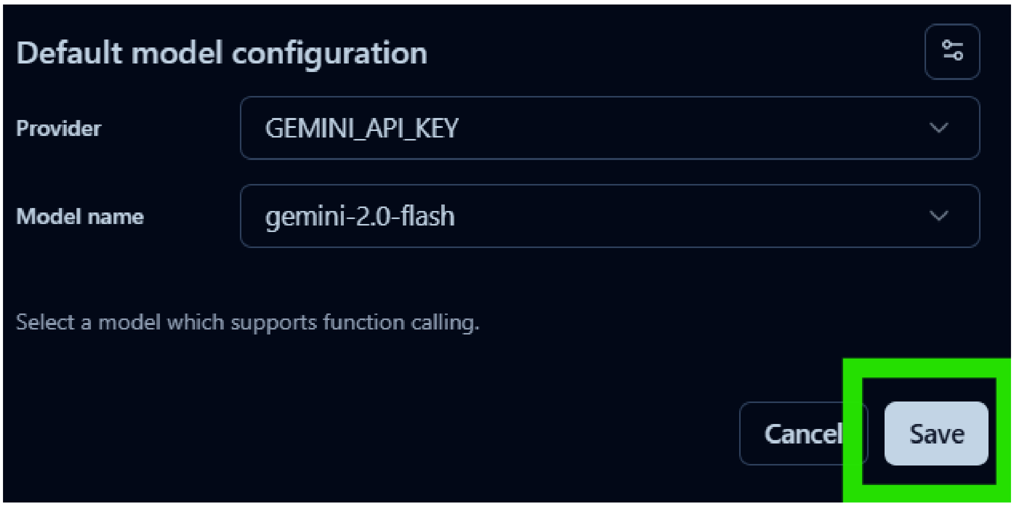



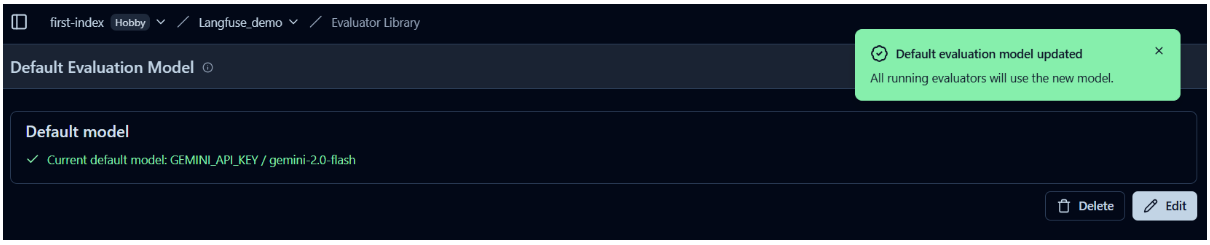

## Setting a Default Model



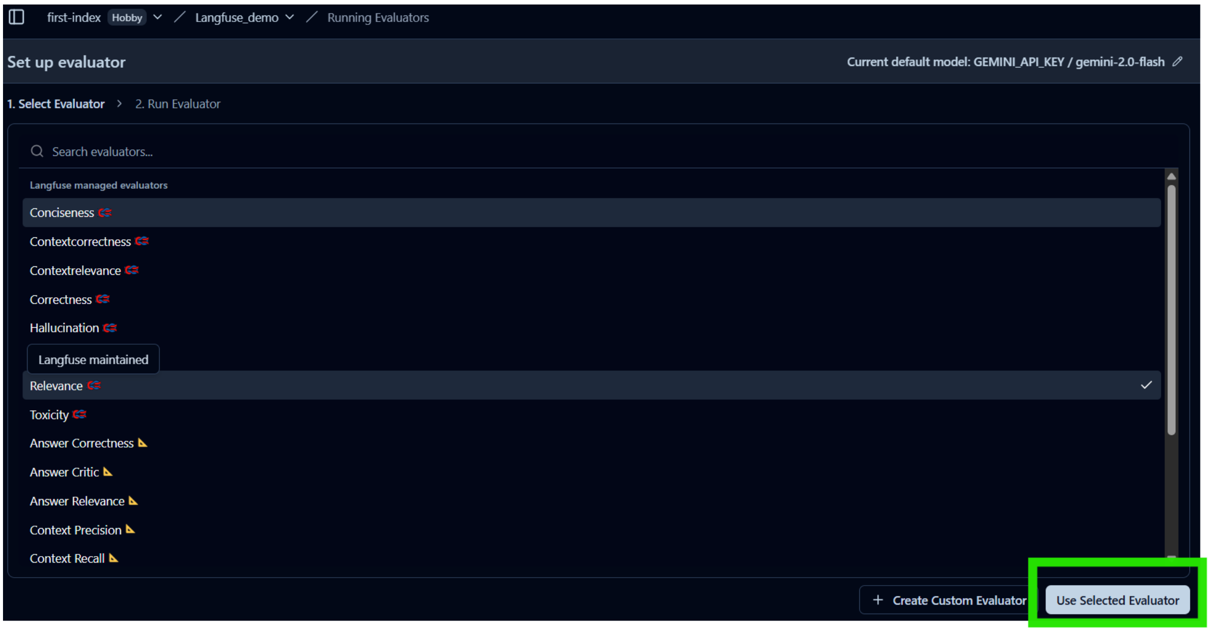

 

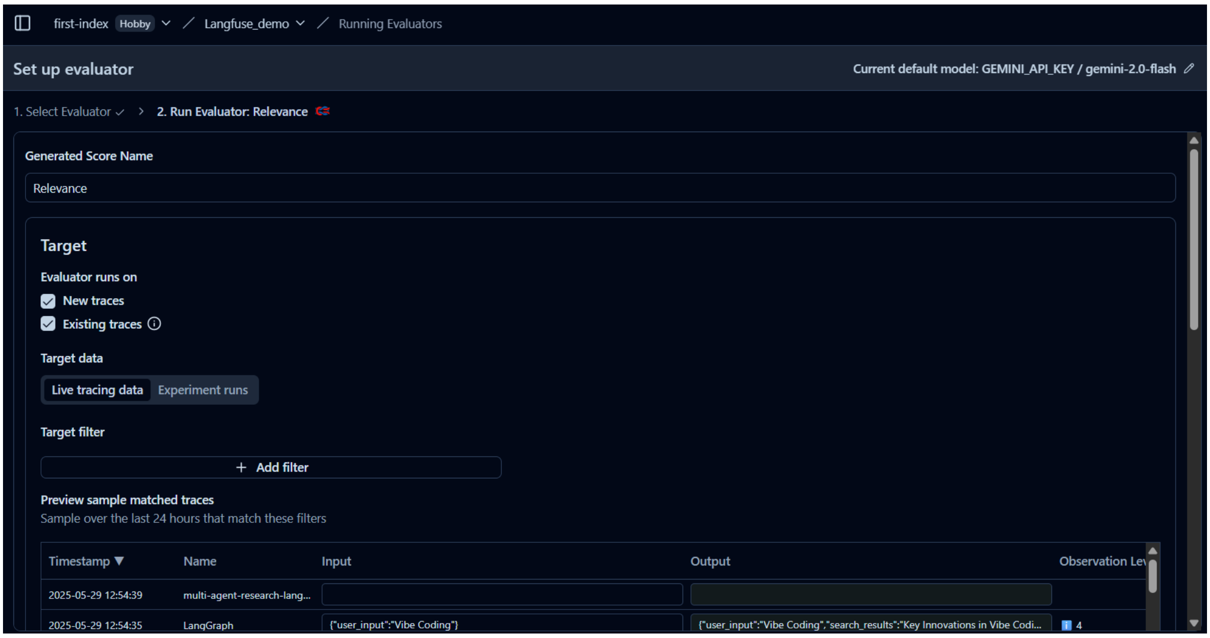


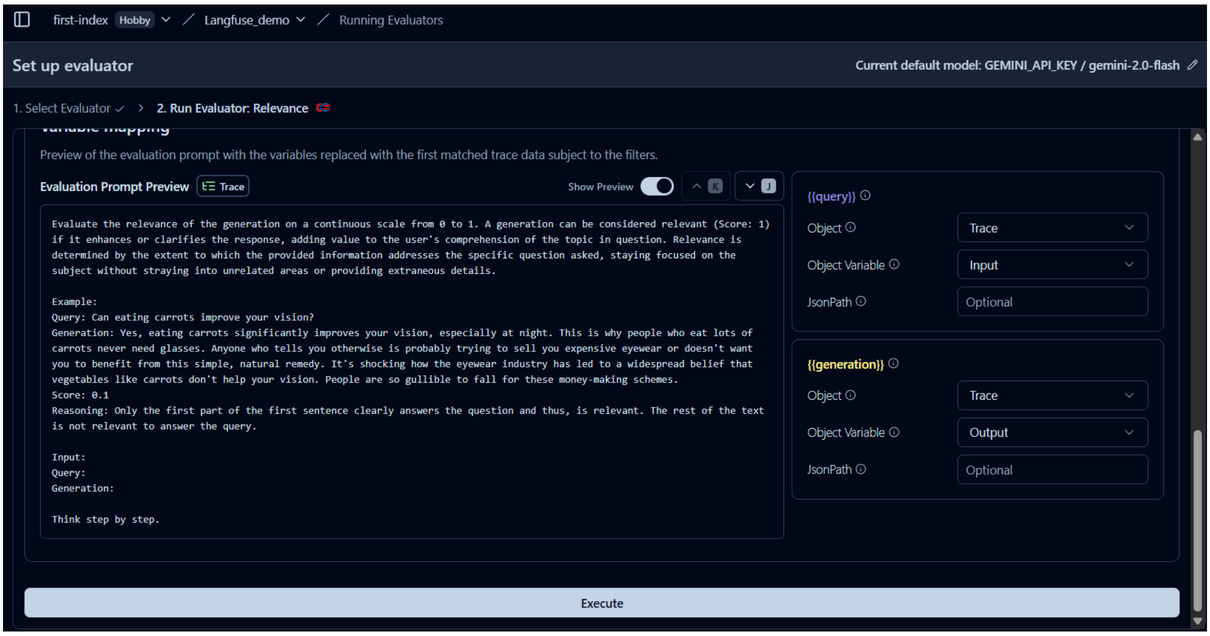




## Use Selected Evaluator






## Active Evaluator after Execution

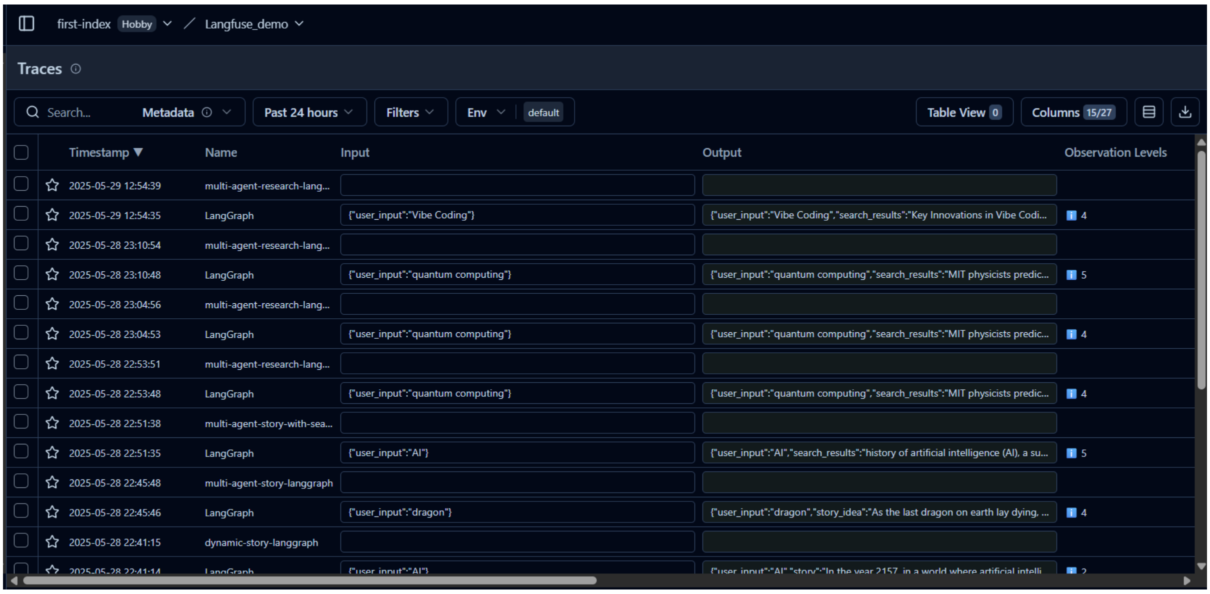
# Evaluator Log View


Evaluator Relevance: 0fc6d50d-9ef8-496c-bae8-6c700b0d2f0d

Filters | Columns 9/9

| Status | Start Time | End Time | Score Name | Score Value | Score Comment | Error | Trace | Template |
|---|---|---|---|---|---|---|---|---|
| pending | 5/29/2025, 1:23:40 PM | | | | | | c4f03cfe-8764-4f54-8a... | cmal6wart005lynrdt |
| pending | 5/29/2025, 1:23:40 PM | | | | | | fd10dca8-26ba-45e1-b... | cmal6wart005lynrdt |
| completed | 5/29/2025, 1:23:40 PM | 5/29/2025, 1:24:11 PM | Relevance | 0 | The generation is an er... | | a8534215-db00-4832-... | cmal6wart005lynrdt |
| completed | 5/29/2025, 1:23:40 PM | 5/29/2025, 1:24:11 PM | Relevance | 0 | The query is empty. Th... | | d27731ad-d8e2-4b1e-8... | cmal6wart005lynrdt |
| pending | 5/29/2025, 1:23:40 PM | | | | | | 2534972e-c01e-40b9-a... | cmal6wart005lynrdt |
| completed | 5/29/2025, 1:23:40 PM | 5/29/2025, 1:24:11 PM | Relevance | 1 | The generation tells a s... | | a366684b-f3e6-46df-a... | cmal6wart005lynrdt |
| pending | 5/29/2025, 1:23:40 PM | | | | | | 6dcdd70b-c018-4841-a... | cmal6wart005lynrdt |
| pending | 5/29/2025, 1:23:40 PM | | | | | | 283d9beb-1c6e-488d-... | cmal6wart005lynrdt |
| completed | 5/29/2025, 1:23:40 PM | 5/29/2025, 1:24:11 PM | Relevance | 0 | The query is empty. Th... | | 74930a3a-9f4e-48a2-a... | cmal6wart005lynrdt |
| pending | 5/29/2025, 1:23:40 PM | | | | | | 9030a19b-29d4-4aa2-... | cmal6wart005lynrdt |
| completed | 5/29/2025, 1:23:40 PM | 5/29/2025, 1:24:11 PM | Relevance | 0.9000 | The generation tells a s... | | 289daa5e-bbcf-470a-a... | cmal6wart005lynrdt |
| completed | 5/29/2025, 1:23:40 PM | 5/29/2025, 1:24:12 PM | Relevance | 0.9500 | The generation provide... | | 9397fc58-a81f-49ab-81... | cmal6wart005lynrdt |
| completed | 5/29/2025, 1:23:40 PM | 5/29/2025, 1:24:11 PM | Relevance | 0 | The query is empty. Th... | | d2710476-e911-4967-a... | cmal6wart005lynrdt |

# Traces:

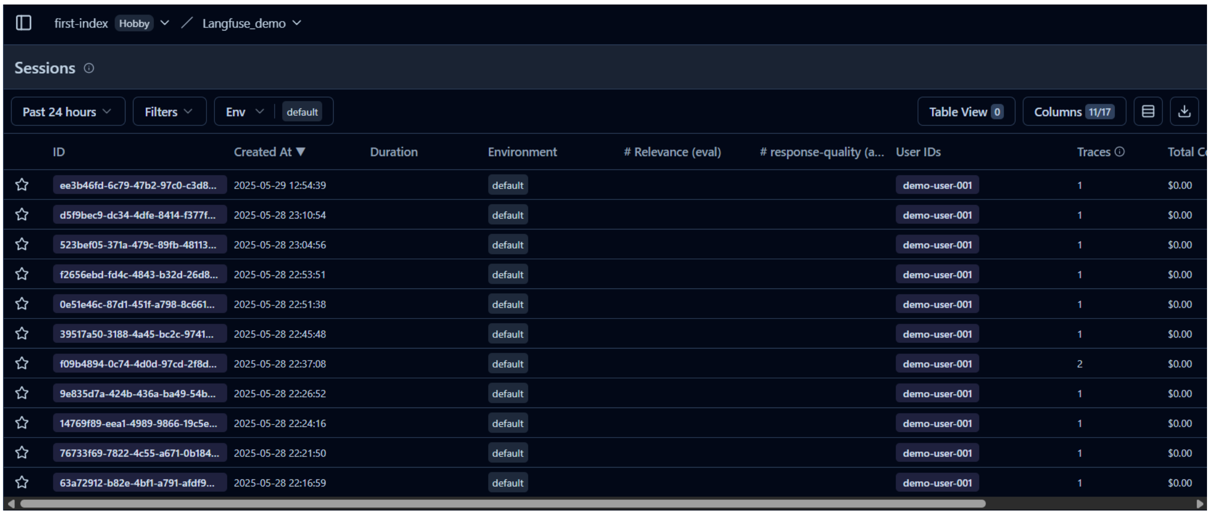

first-index Hobby / Langfuse_demo

Traces

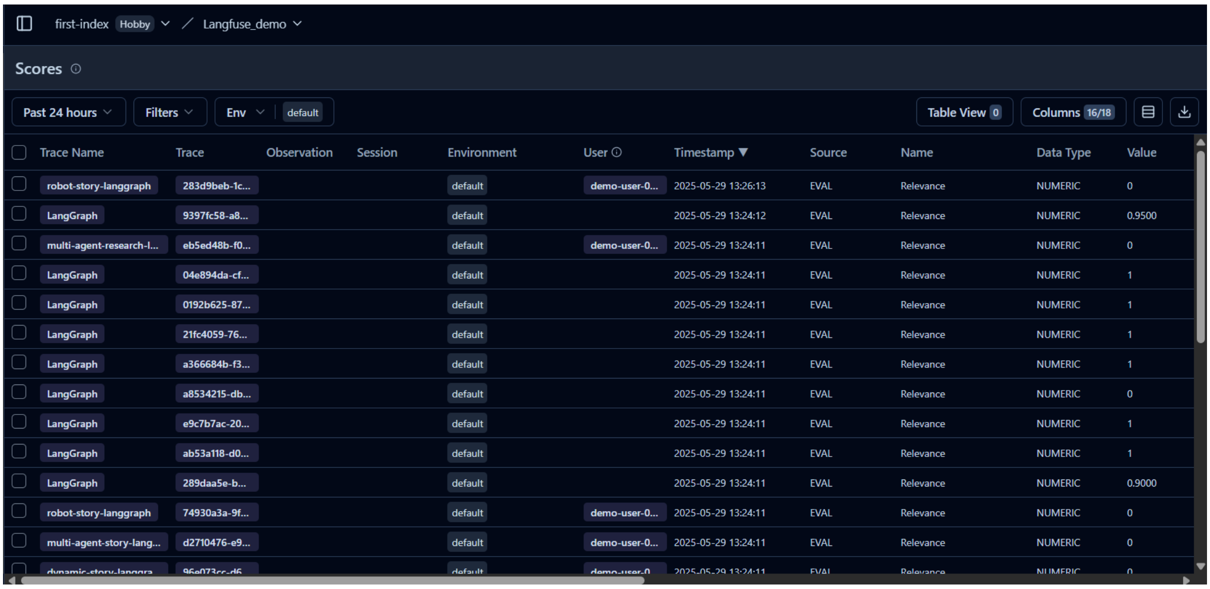
Search... Metadata | Past 24 hours | Filters | Env default | Table View 0 | Columns 15/27

| Timestamp | Name | Input | Output | Observation Levels |
|---|---|---|---|---|
| 2025-05-29 12:54:39 | multi-agent-research-lang... | | | |
| 2025-05-29 12:54:35 | LangGraph | {"user_input":"Vibe Coding"} | {"user_input":"Vibe Coding","search_results":"Key Innovations in Vibe Codi... | 4 |
| 2025-05-28 23:10:54 | multi-agent-research-lang... | | | |
| 2025-05-28 23:10:48 | LangGraph | {"user_input":"quantum computing"} | {"user_input":"quantum computing","search_results":"MIT physicists predic... | 5 |
| 2025-05-28 23:04:56 | multi-agent-research-lang... | | | |
| 2025-05-28 23:04:53 | LangGraph | {"user_input":"quantum computing"} | {"user_input":"quantum computing","search_results":"MIT physicists predic... | 4 |
| 2025-05-28 22:53:51 | multi-agent-research-lang... | | | |
| 2025-05-28 22:53:48 | LangGraph | {"user_input":"quantum computing"} | {"user_input":"quantum computing","search_results":"MIT physicists predic... | 4 |
| 2025-05-28 22:51:38 | multi-agent-story-with-sea... | | | |
| 2025-05-28 22:51:35 | LangGraph | {"user_input":"AI"} | {"user_input":"AI","search_results":"history of artificial intelligence (AI), a su... | 5 |
| 2025-05-28 22:45:48 | multi-agent-story-langgraph | | | |
| 2025-05-28 22:45:46 | LangGraph | {"user_input":"dragon"} | {"user_input":"dragon","story_idea":"As the last dragon on earth lay dying, ... | 4 |
| 2025-05-28 22:41:15 | dynamic-story-langgraph | | | |

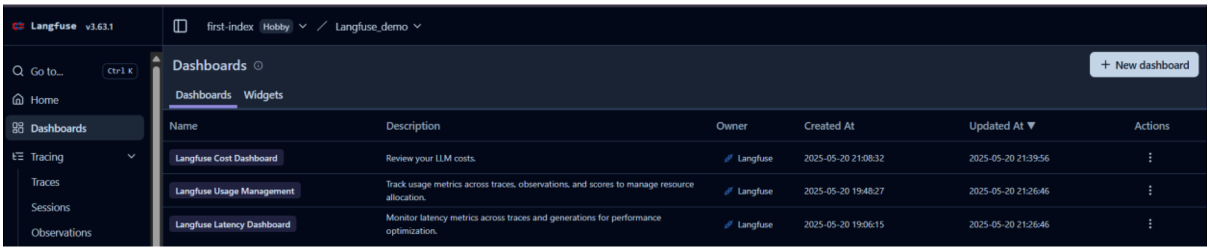
## Sessions:




## Scores:

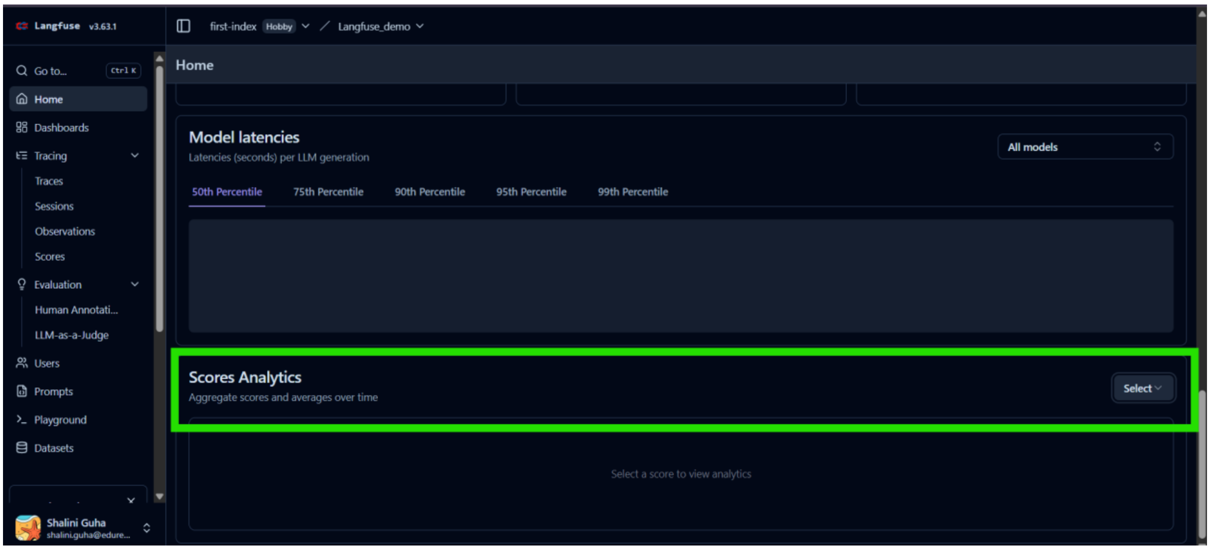



## Dashboards

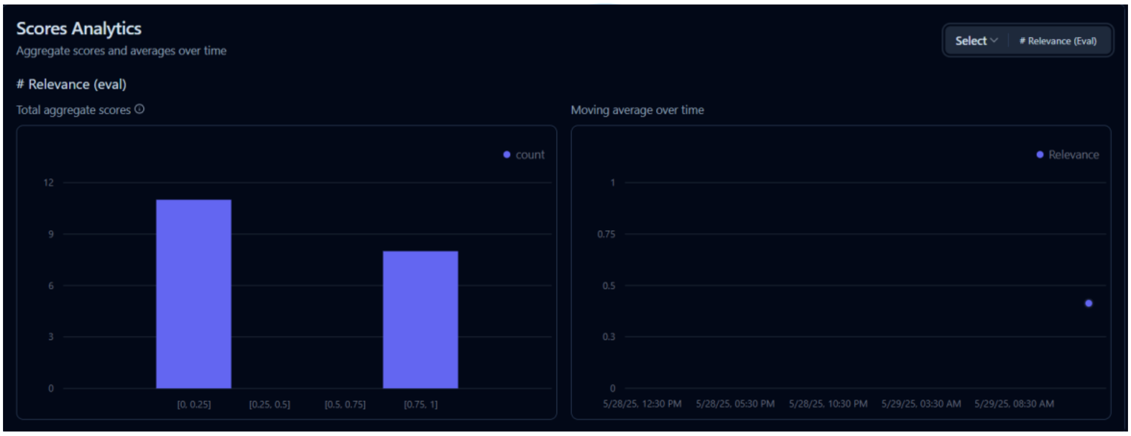


### Project Home

Scroll to the Score Analytics and select the desired analytic to view.




### Relevance Evaluation




### Response Quality